BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 6

Roll no:16 PRN No: 12411696 Name of Student: Tarunesh Pawar

Subject Name : Deep Learning

Problem Statement :

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1. **Start**
2. **Load the Apple Disease Dataset**
3. **Preprocess the images**
4. **Split the dataset**
5. **Apply Data Augmentation**
6. **Build the CNN model**
7. **Flatten the extracted features** into a one-dimensional vector.
8. **Add a Dense Layer** with 128 neurons and ReLU activation.
9. **Apply Dropout (0.5)** to reduce overfitting.
10. **Add the Output Layer**
11. **Compile the CNN**
12. **Train the CNN** using the training dataset for a specified number of epochs.
13. **Validate the model** using the validation dataset.
14. **Evaluate performance** using:
15. **Upload a new apple leaf image.**
16. **Preprocess the uploaded image** in the same way as the training images.
17. **Pass the image through the trained CNN.**
18. **Obtain Softmax probabilities** for all four classes.
19. **Select the class with the highest probability** as the predicted disease.
20. **Display the predicted disease and confidence percentage.**

Code :

```
!pip install -q keras-tuner
import os
import zipfile
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import tensorflow as tf
import keras_tuner as kt

from google.colab import files

from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix

print("TensorFlow:", tf.__version__)
print("Keras Tuner:", kt.__version__)
```

```
!wget -q -O apple-disease.zip \
"https://www.kaggle.com/api/v1/datasets/download/rm1000/apple-disease-
dataset-original"

print("Dataset downloaded successfully!")
```

```
extract_path = "/content/apple_dataset"

os.makedirs(extract_path, exist_ok=True)

with zipfile.ZipFile("/content/apple-disease.zip", "r") as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully!")
```

```
for root, dirs, files_list in os.walk(extract_path):
    level = root.replace(extract_path, "").count(os.sep)
    indent = "  " * level
    print(indent + os.path.basename(root) + "/")

    if level < 2:
        for file in files_list[:5]:
            print(indent + "  " + file)
```

```
image_extensions = (".jpg", ".jpeg", ".png", ".bmp")

class_folders = []

for root, dirs, files_list in os.walk(extract_path):
    image_count = sum(
        file.lower().endswith(image_extensions)
```

```
            for file in files_list
        )

        if image_count > 10:
            class_folders.append(root)

print("Possible class folders:")
for folder in class_folders:
    print(folder)


dataset_dir = None

for root, dirs, files_list in os.walk(extract_path):
    valid_subfolders = []

    for d in dirs:
        folder_path = os.path.join(root, d)

        image_count = 0
        for _, _, fs in os.walk(folder_path):
            image_count += sum(
                f.lower().endswith(image_extensions)
                for f in fs
            )

        if image_count > 10:
            valid_subfolders.append(d)

    if len(valid_subfolders) >= 4:
        dataset_dir = root
        break

print("Dataset directory:")
print(dataset_dir)


class_names = sorted([
    d for d in os.listdir(dataset_dir)
    if os.path.isdir(os.path.join(dataset_dir, d))
])

print("Classes:")
for i, class_name in enumerate(class_names):
    print(i, ":", class_name)


class_counts = {}
```

```
for class_name in class_names:
    folder = os.path.join(dataset_dir, class_name)

    count = 0

    for root, dirs, files_list in os.walk(folder):
        count += sum(
            file.lower().endswith(image_extensions)
            for file in files_list
        )

    class_counts[class_name] = count

print("Number of images per class:\n")

for class_name, count in class_counts.items():
    print(f"{class_name}: {count}")


plt.figure(figsize=(10, 5))

plt.bar(
    class_counts.keys(),
    class_counts.values()
)

plt.xticks(rotation=30, ha="right")
plt.xlabel("Apple Disease Class")
plt.ylabel("Number of Images")
plt.title("Apple Disease Dataset Distribution")

plt.tight_layout()
plt.show()


IMG_SIZE = (128, 128)
BATCH_SIZE = 32
SEED = 42
train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)

class_names = train_ds.class_names

print("Classes:", class_names)


plt.figure(figsize=(12, 8))

for images, labels in train_ds.take(1):
    for i in range(12):
        plt.subplot(3, 4, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.tight_layout()
plt.show()


AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)


data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.15)
])


num_classes = len(class_names)

model = models.Sequential([

    # Input
    layers.Input(shape=(128, 128, 3)),
```

```
    # Data augmentation
    data_augmentation,

    # Normalize pixel values
    layers.Rescaling(1./255),

    # Convolution Block 1
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Convolution Block 2
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Convolution Block 3
    layers.Conv2D(128, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Convert feature maps into vector
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(128, activation="relu"),

    # Reduce overfitting
    layers.Dropout(0.5),

    # Output layer
    layers.Dense(num_classes, activation="softmax")
])

model.summary()
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
EPOCHS = 15

history = model.fit(
    train_ds,
```

```
    validation_data=val_ds,
    epochs=EPOCHS
)


plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Training and Validation Accuracy")
plt.legend()

plt.grid(True)
plt.show()




plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Training and Validation Loss")
plt.legend()

plt.grid(True)
plt.show()
```

```
loss, accuracy = model.evaluate(val_ds)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)


y_true = []
y_pred = []

for images, labels in val_ds:

    predictions = model.predict(
        images,
        verbose=0
    )

    predicted_classes = np.argmax(
        predictions,
        axis=1
    )

    y_true.extend(labels.numpy())
    y_pred.extend(predicted_classes)

y_true = np.array(y_true)
y_pred = np.array(y_pred)

print("Predictions generated successfully!")




cm = confusion_matrix(
    y_true,
    y_pred
)

plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Class")
```

```
plt.ylabel("Actual Class")
plt.title("CNN Confusion Matrix")

plt.xticks(rotation=30, ha="right")
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()


model.save("/content/apple_disease_cnn.keras")

print("Model saved successfully!")


files.download("/content/apple_disease_cnn.keras")


uploaded = files.upload()


from tensorflow.keras.utils import load_img, img_to_array

uploaded_file = list(uploaded.keys())[0]

img = load_img(
    uploaded_file,
    target_size=IMG_SIZE
)

img_array = img_to_array(img)

img_array = np.expand_dims(
    img_array,
    axis=0
)

predictions = model.predict(img_array)

predicted_index = np.argmax(predictions[0])

predicted_class = class_names[predicted_index]

confidence = predictions[0][predicted_index] * 100

plt.figure(figsize=(6, 6))

plt.imshow(img)
```

```
plt.axis("off")

plt.title(
    f"Prediction: {predicted_class}\n"
    f"Confidence: {confidence:.2f}%"
)

plt.show()

print("Predicted Disease:", predicted_class)
print(f"Confidence: {confidence:.2f}%")

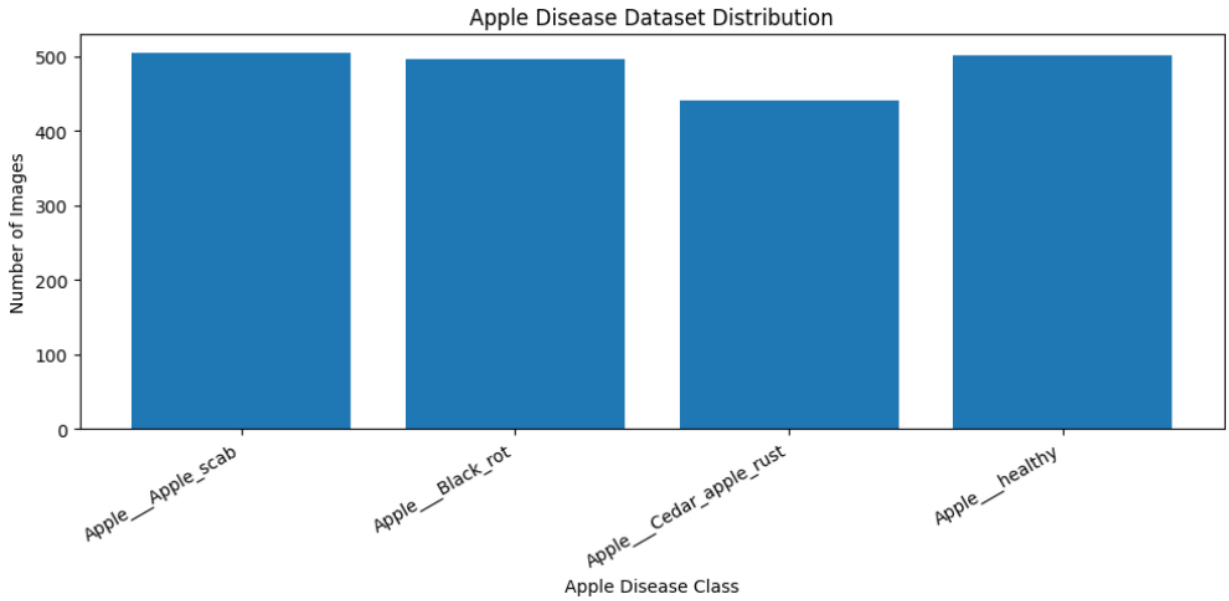



print("Prediction Probabilities:\n")

for class_name, probability in zip(
    class_names,
    predictions[0]
):
    print(
        f"{class_name}: "
        f"{probability * 100:.2f}%"
    )
```

Output :

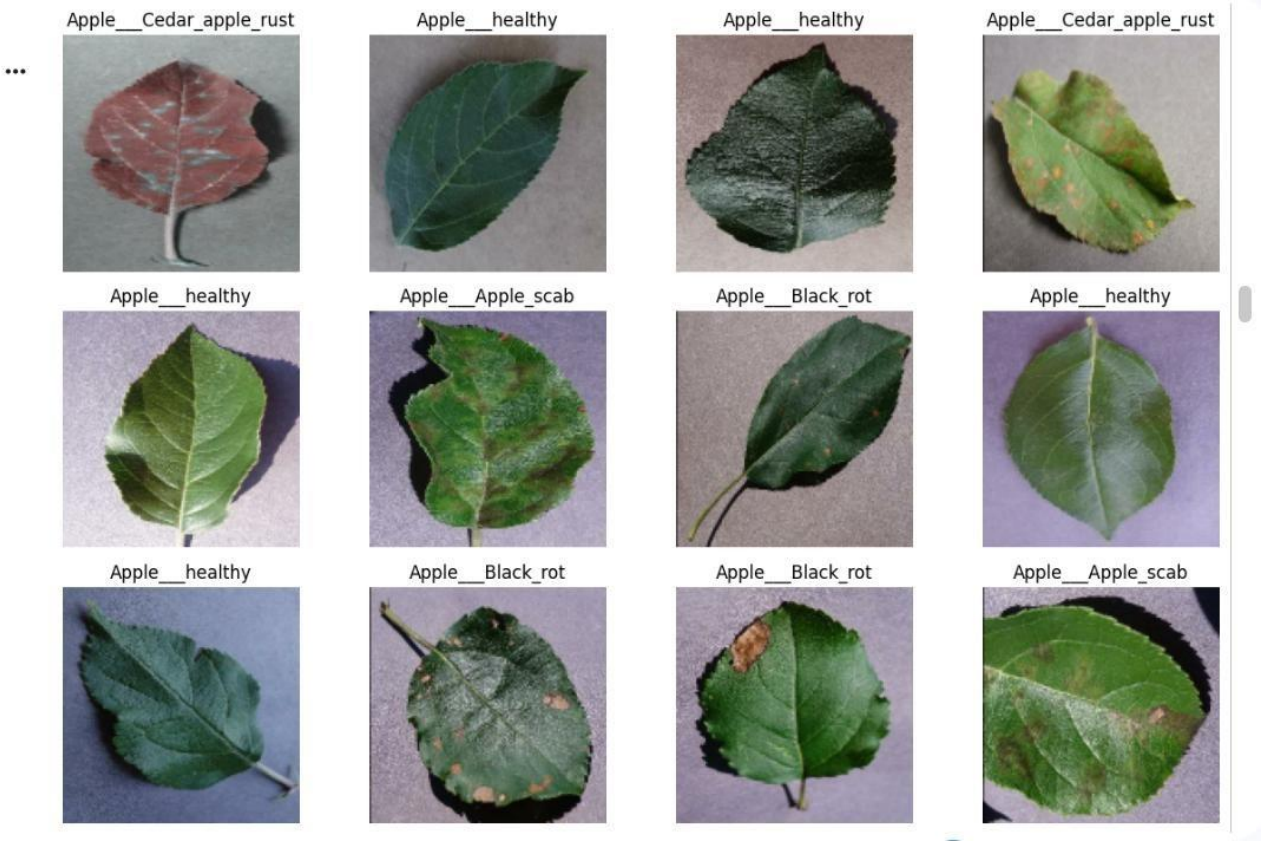
```
plt.show()
```




```
plt.figure(figsize=(12, 8))

for images, labels in train_ds.take(1):
    for i in range(12):
        plt.subplot(3, 4, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.tight_layout()
plt.show()
```



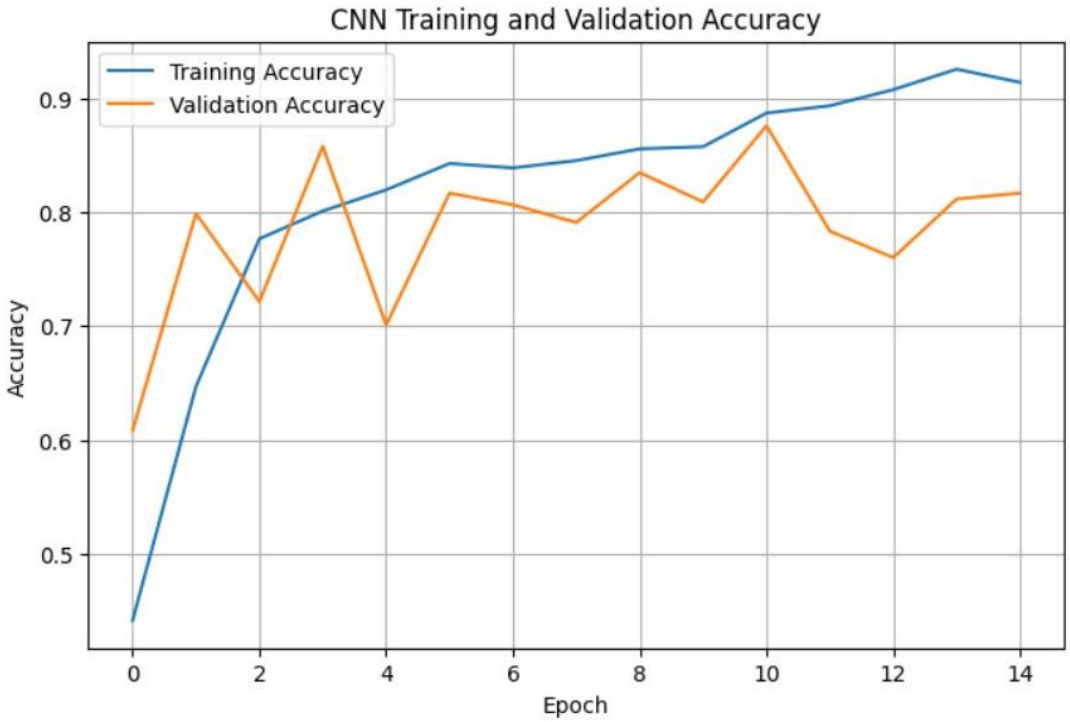

```
model.summary()
```

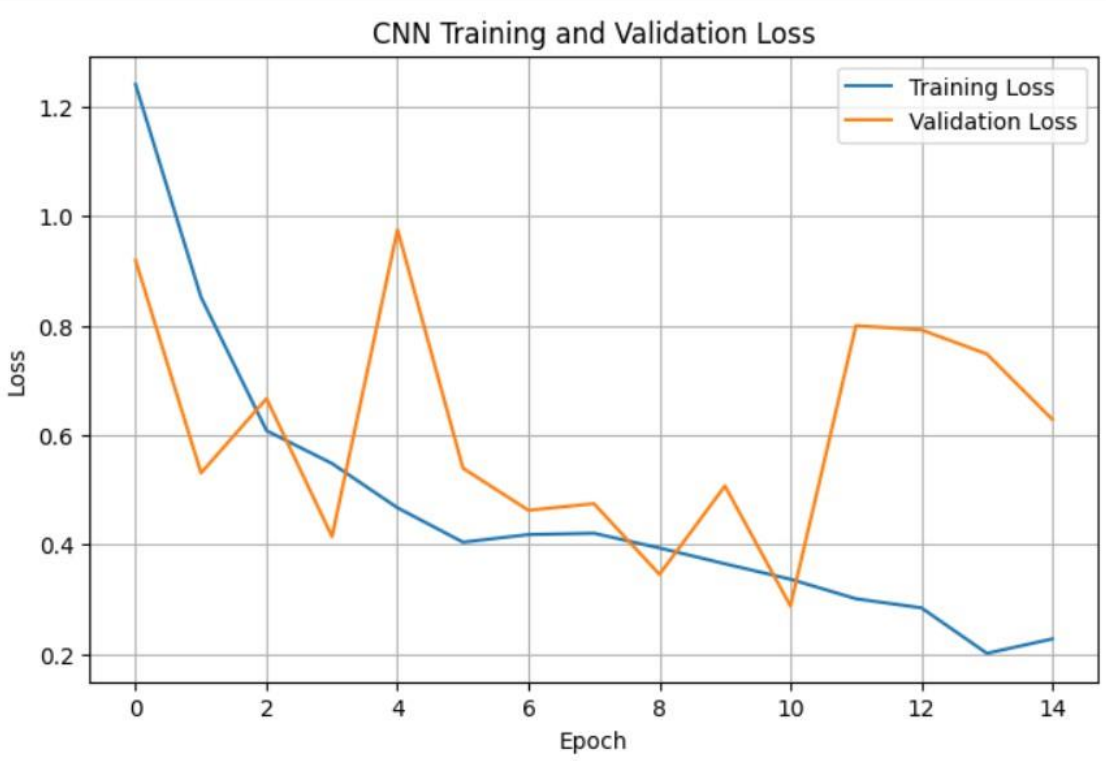
Model: "sequential_1"

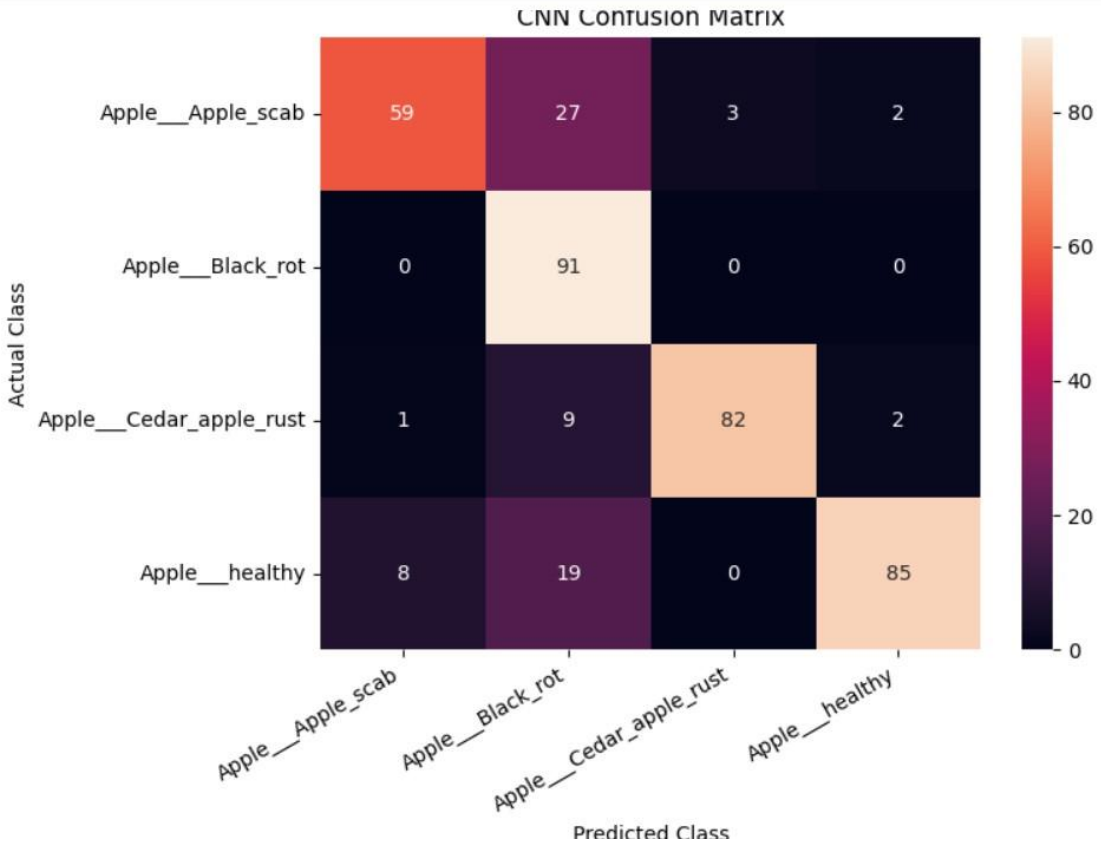
Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 128, 128, 3)	0
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 4)	516

Total params: 3,305,156 (12.61 MB)
Trainable params: 3,305,156 (12.61 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - accuracy: 0.4412 - loss: 1.2409 - val_accuracy: 0.6082
Epoch 2/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 64s 1s/step - accuracy: 0.6469 - loss: 0.8520 - val_accuracy: 0.7990
Epoch 3/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 80s 1s/step - accuracy: 0.7768 - loss: 0.6083 - val_accuracy: 0.7216
Epoch 4/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.8013 - loss: 0.5482 - val_accuracy: 0.8582
Epoch 5/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 64s 1s/step - accuracy: 0.8199 - loss: 0.4673 - val_accuracy: 0.7010
Epoch 6/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 63s 1s/step - accuracy: 0.8431 - loss: 0.4045 - val_accuracy: 0.8170
Epoch 7/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 64s 1s/step - accuracy: 0.8392 - loss: 0.4183 - val_accuracy: 0.8067
Epoch 8/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 64s 1s/step - accuracy: 0.8457 - loss: 0.4208 - val_accuracy: 0.7912
Epoch 9/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 90s 1s/step - accuracy: 0.8559 - loss: 0.3939 - val_accuracy: 0.8351
Epoch 10/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 63s 1s/step - accuracy: 0.8579 - loss: 0.3650 - val_accuracy: 0.8093
Epoch 11/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 86s 1s/step - accuracy: 0.8875 - loss: 0.3368 - val_accuracy: 0.8763
Epoch 12/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 92s 2s/step - accuracy: 0.8939 - loss: 0.3011 - val_accuracy: 0.7835
Epoch 13/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 63s 1s/step - accuracy: 0.9080 - loss: 0.2845 - val_accuracy: 0.7603
Epoch 14/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9260 - loss: 0.2014 - val_accuracy: 0.8119
Epoch 15/15
49/49 ━━━━━━━━━━━━━━━━━━━━ 62s 1s/step - accuracy: 0.9145 - loss: 0.2277 - val_accuracy: 0.8170
```




CNN Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch
1.2
1.0
0.8
0.6
0.4
0.2
0
2
4
6
8
10
12
14


CNN Confusion Matrix
Actual Class
Predicted Class
Apple___Apple_scab
Apple___Black_rot
Apple___Cedar_apple_rust
Apple___healthy
59
27
3
2
0
91
0
0
1
9
82
2
8
19
0
85
80
60
40
20
0

...




```
Predicted Disease: Apple___Black_rot
Confidence: 100.00%
```